

# Framework Express.js

## Objetivo da Aula

Compreender a utilização do framework Express e dos templates EJS para a construção de aplicativos web tradicionais.

## Apresentação

Hoje, é comum a utilização de servidores minimalistas, que oferecem apenas uma quantidade mínima de recursos para tratar as solicitações HTTP, bem como que podem ser embarcados no sistema e executados de forma independente, diretamente no sistema operacional. No ambiente do NodeJS, o framework Express viabiliza a criação de sistemas que utilizam esse tipo de abordagem estrutural.

Aqui, vamos conhecer as características do Express e como ele incorpora as funcionalidades necessárias para tratar as requisições, de forma robusta, como o uso de codificadores específicos para cada tipo de sistema. Veremos, também, como funciona o modelo de roteamento do express, que permite criar endpoints e associá-los aos métodos HTTP com uma programação simples e flexível.

Em seguida, veremos como criar um aplicativo web tradicional, baseado em páginas, e conheceremos os templates EJS, que facilitam a implementação do design do sistema, ao mesmo tempo que separam a lógica e a interface.

## 1. Características Gerais do Express

Para que possamos responder às solicitações HTTP, precisamos de um servidor adequado, no qual eram utilizados quase que exclusivamente o **Apache** e o Internet Information Service (**IIS**), os principais **Web Servers** do mercado. Também temos diversos **Application Servers**, como JBoss, Glassfish, Payara e Web Sphere, os quais oferecem, além das respostas via HTTP, um ambiente de objetos distribuídos para aplicações corporativas.

Com a evolução das tecnologias e a popularização dos **microsserviços**, é cada vez mais comum o uso de um **servidor embarcado**, no qual não é necessária uma estrutura externa, e o aplicativo já contém um servidor HTTP, com o mínimo de funcionalidades. Entre os exemplos de tecnologias, nesse perfil, podemos citar **Spring Boot** (Java), **Flask** (Python) e, no ambiente do NodeJS, **Express**.

Express é um servidor **minimalista**, ou seja, ele utiliza o mínimo de recursos para executar, viabilizando sua inclusão no sistema que será desenvolvido. Além de não exigir um ambiente de execução externo, pode ser disponibilizado via **Docker**, facilmente, como todo projeto NodeJS, o que faz dele uma tecnologia bastante popular para a criação de microsserviços.

Para começar a implementar nosso projeto de exemplo, crie um diretório vazio, abra com o VS Code e execute os comandos apresentados a seguir, de forma a inicializar o projeto e instalar a dependência do express.

```
npm init -y
npm install express
```

Apesar de termos a mesma possibilidade de responder às requisições HTTP utilizando apenas as bibliotecas do núcleo do NodeJS, com o Express temos uma série de recursos que facilitam o tratamento de rotas, assim como do método HTTP que será aceito. Além disso, a captura de valores enviados na requisição é bem mais simples que no modelo padrão do NodeJS, quando utilizamos o módulo http. Observe algumas diferenças no exemplo seguinte.

```
const app1 = http.createServer((req, res)=>{
  if(req.method==="GET" && req.url.startsWith("/produtos")){
    const codigo = req.url.substring(req.url.lastIndexOf('/')+1);
    obterProduto(codigo).then((p) => {
      if(p){
        res.statusCode = 200; res.end(JSON.stringify(p));
      }
    });
  }
});
```

```
const app2 = express();
app2.get('/produtos/:id', async(req,res) => {
  res.json(await obterProduto(req.params.id)); res.end();
});
```

Analisando as duas implementações, em que a primeira utiliza o módulo http, e a segunda trabalha com Express, no caso do módulo nativo, tratamos **todas** as requisições em apenas um método, exigindo que identifiquemos a rota e o método HTTP no corpo da função de callback, enquanto, para o Express, o tratamento da requisição já tem o método HTTP e a rota definidos na captura de forma **individualizada**. O sistema de **roteamento** do Express é um de seus pontos fortes, pois permite uma leitura clara de endpoints e métodos HTTP, sem a necessidade de verificações em meio ao código da callback para tratamento de requisições.

Além disso, os métodos para envio de resposta, no Express, já preveem o uso de codificadores específicos para formatos conhecidos, como **JSON**, e a obtenção de dados, a partir da requisição, também é feita de forma muito mais organizada – como no caso do atributo **params**, que retorna os identificadores nos segmentos parametrizados, sem a necessidade da utilização de métodos como **substring** ou **split**.

#### Quadro 1: Atributos do Express para recuperação de informações

| Atributo   | Descrição                                                                                                                                                          |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| req.params | Recupera a informação do segmento da rota parametrizado. Por exemplo, para um endpoint <b>/tarefas/:id</b> , a recuperação seria através de <b>req.params.id</b> . |
| req.query  | Recupera a informação a partir da Query String da URL. Por exemplo, para uma URL <b>/tarefas?id=101</b> , a recuperação seria através de <b>req.query.id</b> .     |
| req.body   | Recupera as informações no corpo da requisição. Para o formato JSON, sua utilização é direta, mas a forma de uso depende do codificador utilizado.                 |

Fonte: Elaboração própria.

Agora que vimos algumas vantagens da utilização do Express, vamos criar o arquivo index1.js, com o conteúdo da listagem seguinte.

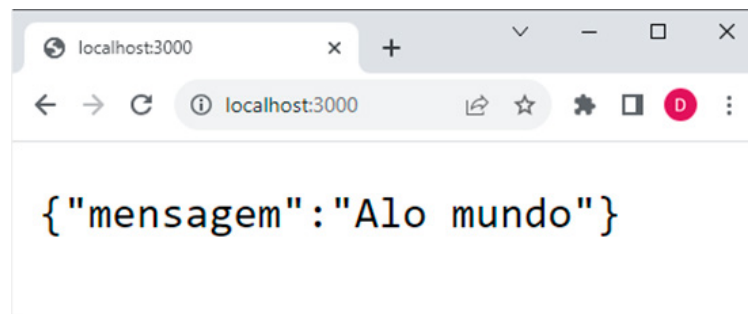
```
const express = require('express');
const app = express();
const port = 3000;
```

```
app.get('/', (req, res) => {
  res.json({mensagem: "Alo mundo"});
  res.end();
})
app.listen(port, () => {
  console.log(`Servidor executando na porta ${port}`);
})
```

Através desse código, nós definimos uma rota **raiz**, que responde ao método **GET** do HTTP, em que, na resposta, foi utilizado o formato **json**, com a passagem de um objeto com a mensagem. Note a simplicidade para especificar o método HTTP e o tratamento direto de elementos JSON, o que o torna muito superior ao módulo padrão http, no qual teríamos que transformar o objeto em texto para o retorno, além de testar o método utilizado no corpo da função.

Execute o projeto usando o comando **node index1**. Após executar o servidor, acesse-o através do navegador ou de alguma ferramenta de teste.

**Figura 1:** Acesso à rota raiz pelo navegador



Fonte: Elaboração própria.



### Você Sabia?

Da mesma forma que temos o framework Express substituindo o módulo http no lado servidor, no lado cliente você pode utilizar a biblioteca **Axios**, que simplifica muito a criação de requisições HTTP.

Certamente, o que chama muito a atenção no Express é o **roteamento**, muito robusto e flexível. Ele define os **endpoints**, associando com o **método HTTP** permitido, além de oferecer um sistema de parametrização que deixa mais simples a recuperação dos valores nos endereços dinâmicos. Também já define um tratamento padrão para erros como **404**, quando a rota não é identificada.

Outra característica interessante do Express é a possibilidade de trabalhar com **View Engines**, ou **motores de visualização**, nos quais o tratamento da rota apenas recupera os valores e repassa para um **template**, responsável pela construção da resposta, dividindo o processamento entre a parte lógica e o design. Essa é uma técnica muito adotada nos sistemas web clássicos, em que o HTML define a interface de usuário e diversas plataformas oferecem motores de visualização, como o **Thymeleaf**, para Java, e o Embedded JavaScript (**EJS**), utilizado pelo NodeJS.

## 2. Tratamento de GET e POST

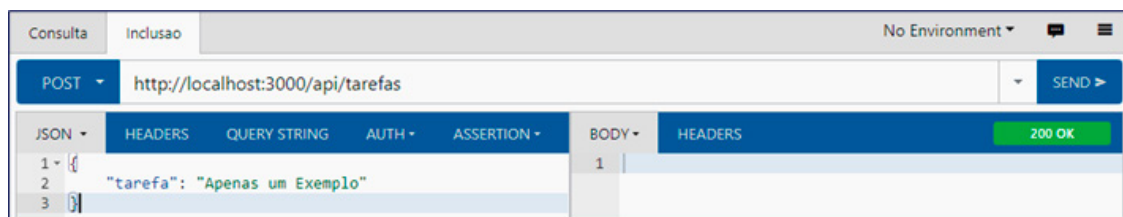
Na arquitetura REST, os métodos GET e POST são utilizados para consulta e criação de recursos, respectivamente. Para demonstrar o uso, vamos criar o arquivo **index2.js**, em nosso projeto, e utilizar o conteúdo da listagem seguinte.

```
const express = require('express');
const app = express();
app.use(express.json());
let tarefas = [];
app.get('/api/tarefas', (req, res) => {
  res.json(tarefas);
  res.end();
})
app.post('/api/tarefas', (req, res) => {
  tarefas.push({ tarefa: req.body.tarefa });
  res.end();
})
app.listen(3000);
```

Observando a listagem, temos uma instância de Express no objeto **app**, e a primeira configuração efetuada foi o **uso de codificação JSON**. Além da escolha do codificador, temos um mesmo **endpoint** respondendo via **GET** e **POST**, em que a callback do método **get** apenas retorna a lista de tarefas atual, e, no método **post**, a callback acrescenta uma tarefa recebida no formato JSON à lista, com a captura do valor, a partir de **req.body**, fornecendo uma resposta vazia ao final.

Para testar o servidor, execute o comando **node index2** e utilize alguma ferramenta para efetuar as requisições, como **Postman** ou **Boomerang**.

**Figura 2:** Inclusão de tarefa via Boomerang



*Fonte: Elaboração própria.*

Os aplicativos web tradicionais são baseados em **páginas**, e suas interfaces cadastrais utilizam **formulários** HTML, trabalhando apenas com os métodos GET e POST. Isso é bem diferente do fornecimento de uma API, pois, aqui, existe o desenho da interface de usuário, exigindo que sejam construídos tanto o back-end quanto o front-end no mesmo sistema.

Vamos iniciar a criação da terceira versão para nosso servidor, no arquivo **index3.js**, com o conteúdo da listagem seguinte.

```
const express = require('express');
const app = express();
app.use(express.urlencoded({ extended: true }));
let tarefas = [ {tarefa: "compilar"}, {tarefa: "testar"} ];
const getPagina = () => {
  let conteudo = "<html><body><form method='post'>";
  conteudo += "<input type='text' name='tarefa'";
  conteudo += " placeholder='Tarefa'>";
  conteudo += "<input type='submit'";
```

```
conteudo += " value='Adicionar'></form><ul>";
for(let t of tarefas)
  conteudo += `<li>${t.tarefa}</li>`;
conteudo += "</ul></body></html>";
return conteudo;
}

app.get( [ '/', '/web/tarefas' ], (req, res) => {
  res.status(200)
    .contentType('text/html')
    .send(getPagina());
})

app.post( [ '/', '/web/tarefas' ], (req, res) => {
  tarefas.push({tarefa: req.body.tarefa})
  res.status(200).contentType('text/html').send(getPagina());
})

app.listen(3000)
```

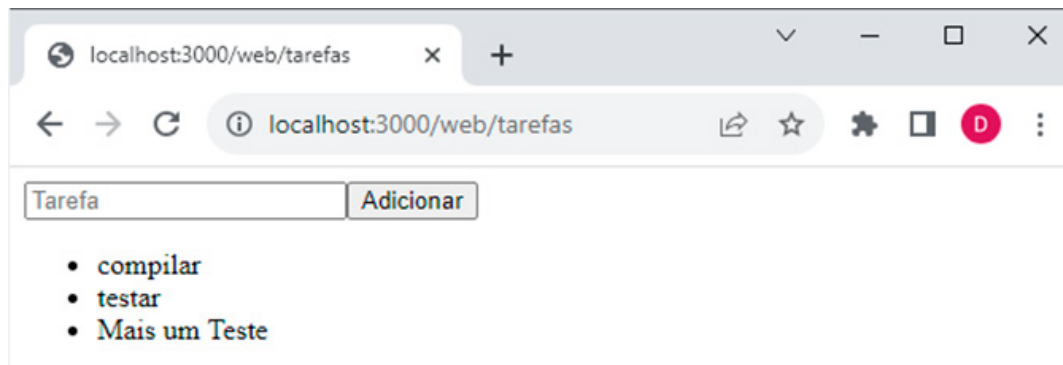
A primeira grande mudança aqui é a do **codificador**, pois agora recebemos os dados a partir de um formulário, exigindo **urlencoded**. A captura dos valores enviados é similar à utilizada no REST, através do atributo **body** da requisição.

Para os aplicativos web tradicionais, o retorno de uma requisição HTTP é uma página HTML, e, para diminuir o esforço de programação, criamos a função **getPagina**, que constrói a página de retorno, com o preenchimento de uma lista HTML com as tarefas. A página gerada também oferece um formulário com método POST, que, por não especificar a ação, chama a rota corrente.

As respostas oferecidas via GET e POST são semelhantes, mas, ao tratar a chamada POST, que ocorre a partir do formulário, temos a inclusão da tarefa que foi enviada no repositório local. Note que temos **duas** rotas viáveis para cada um dos métodos, em que o conjunto de endpoints fica em um vetor.

Para fazer um teste, execute o comando **node index3** e acesse o endereço **http://localhost:3000/web/tarefas** através de um navegador.

**Figura 3:** Servidor no modelo tradicional



Fonte: Elaboração própria.



### Destaque

Se o formulário utilizasse **GET**, os dados estariam disponíveis na Query String, sendo recuperados a partir do atributo **query** da requisição.

## 3. Utilizando Templates EJS

Ao trabalhar no modelo tradicional para web, fica fácil perceber que temos uma grande fragilidade na construção da resposta, pois, além de exigir extensos trechos de código, mescla a sintaxe HTML com o JavaScript, dificultando muito a manutenção. Mais que isso, deixa a responsabilidade do design nas mãos do programador, enquanto poderia ser utilizado um profissional mais adequado na construção da interface, com ferramentas de maior produtividade.

Uma solução elegante para esse problema é a adoção de **templates HTML**, que podem ser editados livremente pelos designers e preenchidos com os dados que são fornecidos pela camada de lógica, a qual é implementada em JavaScript no ambiente do NodeJS. Quem faz a substituição dos dados no template HTML, em tempo de execução, é um motor de visualização, e aqui iremos utilizar o Embedded JavaScript Templates (EJS).

Vamos compreender seu funcionamento, de forma prática, começando com a instalação do EJS, em nosso projeto, através do comando seguinte.

```
npm install ejs
```



Em seguida, crie o diretório **views**, no projeto; dentro dele, crie um arquivo template, com o nome **exemplo.ejs**, no qual será utilizado o código seguinte.

```
<html><body>
<table border="1">
<% for(let obj of pessoas) { %>
<tr><td><%=obj.nome%></td>
<td><%=obj.idade%></td></tr>
<% } %>
</table>
</body></html>
```

Temos uma página HTML, apesar da extensão **ejs**, com alguns trechos que terão substituição ao nível do servidor. São aceitas duas formas de interação:

- Blocos executáveis, iniciados com **<%** e terminados com **%>**;
- Blocos de substituição de valor, iniciados com **<%=** e terminados com **%>**.

Analisando a parte dinâmica do código, é executado um loop do tipo for, que engloba a linha da tabela através do uso de chaves, criando uma linha para cada iteração do loop. São percorridos todos os objetos de uma coleção de **pessoas**, fornecida por um **controlador**, e, para cada linha gerada, substitui-se o conteúdo interno das células pelos valores do nome e da idade da pessoa corrente.

Agora, vamos criar o arquivo **index4.js** e utilizar o seguinte conteúdo.

```
const express = require('express')
const app = express()
app.set('view engine', 'ejs');
app.set('views', './views');
app.get('/exemplo', (req, res) => {
  const dados = [{nome: "Carlos", idade: 45},
    {nome: "Joana", idade: 36}];
```

```
res.render('exemplo', { pessoas: dados });  
});  
app.listen(3000);
```

No código, temos a configuração de **view\_engine** como **ejs** e a definição do diretório de templates através do parâmetro **views**. Em seguida, implementamos o **controlador** que alimentará o template, respondendo no endereço **/exemplo** via método **get**.

É muito importante observar como ocorre a ligação entre o controlador e o template, com a utilização do método **render**. Tudo que precisamos informar é o **nome do template** EJS e os **dados** que serão enviados para ele, no formato JSON, sendo fornecido, no exemplo, o vetor **pessoas**.

Para executar o servidor, utilize o comando **node index4**, e o teste pode ser feito com o acesso ao endereço **http://localhost:3000/exemplo**, através de um navegador. Você deverá ter, como resultado, os dados das pessoas de exemplo em uma tabela HTML.

## 4. Aplicativo Web Tradicional

Para um exemplo prático de utilização das tecnologias aqui descritas, vamos criar um aplicativo web tradicional, aproveitando o projeto corrente. O objetivo é apenas criar uma página para listagem, inclusão e exclusão de contatos.

Vamos começar pelo controlador, no arquivo **index5.js**, utilizando o conteúdo da listagem seguinte.

```
const express = require('express')  
const app = express()  
const port = 3000;  
app.set('view engine', 'ejs');  
app.set('views', './views');  
app.use(express.urlencoded({ extended: true }));  
let contatos = [{nome: "XPT0", email: "xpto@xpto.com"}];  
app.get('/contatos', (req, res) => {  
  if(req.query.delId)  
    contatos.splice(req.query.delId,1);
```

```
res.render('contatos', {contatos: contatos});
})
app.post('/contatos', (req, res) => {
contatos.push({nome: req.body.n1, email: req.body.e1});
res.render('contatos', {contatos: contatos});
})
app.listen(port);
```

Inicialmente, temos a configuração do Express para utilizar EJS e codificação para captura de formulário (urlencoded), além do repositório local **contatos**, com um contato de exemplo. Em seguida, temos o tratamento do endpoint **/contatos** para os métodos **GET** e **POST** do HTTP.

Para o método **POST**, os valores de **req.body**, que devem ser fornecidos a partir de um **formulário**, são incluídos no repositório local no formato JSON. Em seguida, com base no método **render**, o template EJS será invocado, utilizando os dados do repositório na variável **contatos**, e construirá a resposta.

Já na chamada com método **GET**, caso exista um parâmetro **delId** na Query String, ele será utilizado para remover a posição fornecida do repositório local, via método **splice**. Ocorrendo remoção ou não, a resposta é construída por meio do template, da mesma forma que ocorria para o método POST.



### Destaque

Uma chamada para **http://localhost:3000/contatos?delId=2**, via método **GET**, em nosso sistema, iria remover o terceiro contato da lista.

Falta apenas definir o template, que deve ser criado no diretório **views**, com o nome **contatos.ejs**, utilizando o código apresentado a seguir.

```
<html><body>
<form method="post">
<input type="text" name="n1"><br/>
```

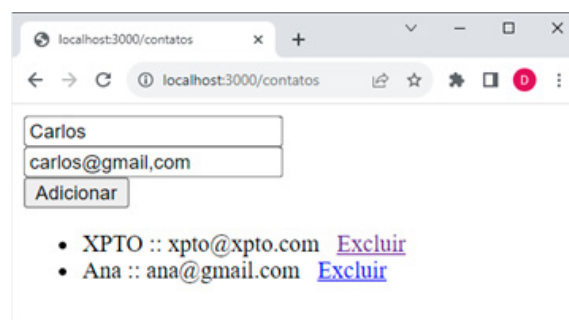
```
<input type="email" name="e1"><br/>
<button type="submit">Adicionar</button>
</form><ul>
<% for(let i=0; i<contatos.length; i++) {%>
<li><%= contatos[i].nome %>:: <%= contatos[i].email %>
&nbsp; <a href="/contatos?delId=<%=i%>">Excluir</a></li>
<% } %>
</ul>
</body></html>
```

Nosso template tem um formulário na parte inicial, no qual o envio ocorre para a rota corrente via método **POST**. Os campos fornecidos são nomeados como **n1** e **e1**, sendo recuperados no servidor através de **req.body**.

Em seguida, utilizamos um **for** tradicional, com **i** variando de zero até a última posição do vetor **contatos**, fornecido pelo **controlador**, para definir as linhas de uma **lista HTML**. Em cada linha são apresentados os atributos **nome** e **email** da posição **i**, além de criar um link para exclusão, dinamicamente, em que **delId** recebe o valor de **i** para a posição a ser removida.

Agora é só executar o servidor, através do comando **node index5**, e testar o sistema no endereço **http://localhost:3000/contatos**. O resultado da execução é apresentado na figura seguinte.

**Figura 4:** Aplicativo web com Express e EJS



*Fonte: Elaboração própria.*

## Considerações Finais da Aula

Os servidores minimalistas são uma estratégia de desenvolvimento cada vez mais comum, estando presente em plataformas como: Python, com o Flask; Java, em que temos o Spring Boot; e NodeJS, que oferece o framework Express. São produtos similares, oferecendo o tratamento para requisições HTTP para todos os métodos do protocolo, além de permitirem, em maior ou menor escala, o uso de funcionalidades mais avançadas, como codificadores e autenticação OAuth.

Para um sistema web tradicional, baseado em páginas, o requisito mínimo é o tratamento de rotas via métodos GET e POST, algo que o Express oferece, de forma simples, robusta e flexível, através de seu modelo de roteamento.

Para facilitar a criação desse tipo de sistema, o uso de templates baseados em HTML é um grande facilitador, além de separar a lógica e o design de telas, permitindo especializar o desenvolvimento por perfil de profissional.

Com os conhecimentos adquiridos, fomos capazes de criar um aplicativo web simples, mas que incorpora as técnicas necessárias para a definição de sistemas maiores, nos quais teríamos diversas páginas, mas continuaríamos a utilizar apenas os métodos GET e POST do protocolo HTTP.

## Materiais Complementares



### **Express - Framework web rápido, flexível e minimalista para Node.js**

2023, Express.

O site oficial do Express oferece, além de informações para instalação, uma boa documentação do produto, com diversos exemplos de utilização.

Link para acesso: <https://expressjs.com/pt-br/> (acesso em: 13 set. 2023).



### **EJS - Embedded JavaScript templating**

2023, EJS.

O site oficial do EJS oferece, além de informações para instalação, uma boa documentação do produto, com diversos exemplos de utilização.

Link para acesso: <https://ejs.co/> (acesso em: 13 set. 2023).

## Referências

FLANAGAN, D. **JavaScript: o guia definitivo**. Porto Alegre: Bookman, 2014. Disponível em: <https://abre.ai/gKdo>. Acesso em: 12 set. 2023.

OLIVEIRA, C; ZANETTI, H. **JavaScript descomplicado: programação para a Web, IoT e dispositivos móveis**. São Paulo: Érica, 2020. Disponível em: <https://abre.ai/gKdq>. Acesso em: 12 set. 2023.

OLIVEIRA, C; ZANETTI, H. **Node.js: programe de forma rápida e prática**. São Paulo: Expressa, 2021. Disponível em: <https://abre.ai/gKdt>. Acesso em: 12 set. 2023.